RedHat EX415 Exam Preparation

1. Preface

This document has been created as a study and reference guide to assist in preparing for the **Red Hat Certified Specialist in Security: Linux (EX415)** exam. It goes through the official exam objectives and covers key topics to the best of my knowledge and understanding, based on independent research and practical hands-on experience.

Please note that this document is intended solely as a self-study aid and supplementary resource. It should not be considered a complete or 100% guarantee for passing the EX415 exam. Candidates are strongly encouraged to consult official Red Hat documentation, complete hands-on labs, and review updated blueprint guidelines during their preparation.

Scope & Roadmap

This guide focuses on core competencies required to harden Red Hat Enterprise Linux, in line with the primary EX415 objectives, this volume places a strong emphasis on:

Ansible Automation: Automating configuration management, security remediation, and compliance scaling.

SELinux: Managing policies, troubleshooting denials, and applying advanced confinement techniques.

System Hardening & Auditing: Configuring the Linux Audit system, file integrity monitoring with AIDE, and restricting USB access via USBGuard.

Data Protection & Access Controls: Implementing LUKS storage encryption, Network-Based Device Encryption (NBDE), and Pluggable Authentication Modules (PAM).

Compliance & Vulnerability Management: Evaluating and remediating systems using OpenSCAP and Red Hat Insights.

Disclaimer

The author assumes no responsibility or liability for any misconfigurations, data loss, system downtime, or security incidents resulting from the application of the concepts described in this document. Always ensure you have explicit authorization, proper backup procedures, and a controlled test environment before applying security policies, encryption, or hardening scripts to production systems.

Table of Contents

1. Preface....1
    Scope & Roadmap....1
    Disclaimer....1
2. Prerequisites & Lab Setup....3
    Host & Virtualization....3
    Lab Topology....3
3. SELinux....4
    SELinux Operating Modes....4
    SELinux Labels (Contexts)....5
    Httpd Examples....8
    SELinux Logs and Troubleshooting....10
    SELinux Booleans....12
    SELinux Port Contexts....13
    SELinux Architecture: SELinux User and Roles (RBAC)....14
4. USBGuard....15
    USBGuard Rule-Set Documentation....17
    Ansible USBGuard Deployment....18
5. PAM Basics....19
    pwquality....20
    Faillock....21
6. Auditd....22
    Auditd Examples....23
7. AIDE....23
    AIDE R Macro and Custom Rules Summary....25
8. Data, Storage, File Systems and Encryption....27
    Storage....27
    Partition tables....27
    GPT Structure....28
    Filesystem....28
    Hard and Soft Links....28
    Data encryption....29
9. OpenScap....32

2. Prerequisites & Lab Setup

Before diving into security configurations, hardening, and automation, a properly configured laboratory environment is essential. This guide assumes that the reader is already proficient in setting up virtual machines, configuring basic networking, and managing Linux systems via the command line independently.

Host & Virtualization

Host OS: Windows 11 (64-bit)

Hypervisor: VMware Workstation Pro (17.)

Note: VMware Workstation Pro is now available for free for personal use via the Broadcom portal. A free Broadcom account registration is required prior to downloading.

Lab Topology

Our lab environment consists of three virtual machines running **AlmaLinux 10** to simulate a infrastructure, utilizing an Ansible control node and managed nodes for configuration management and automated security hardening:

Ansible Control Node (alma10)

OS: AlmaLinux 10

Role: Central management station used to run Ansible playbooks, manage configurations, and execute compliance scans across the managed nodes.

Managed Node 1 (almaserver01)

OS: AlmaLinux 10

Role: Primary target system for applying security hardening policies, SELinux configurations, auditing, and storage encryption.

Managed Node 2 (almaserver02)

OS: AlmaLinux 10

Role: Secondary target system used for testing multi-node compliance scaling, OpenSCAP remediation, and role-based access controls.

3. SELinux

Security-Enhanced Linux (SELinux) is a mandatory access control (MAC) architecture integrated directly into the Linux kernel. Unlike traditional Discretionary Access Control (DAC), where file permissions (rwx) and ownership are controlled solely by users or file owners, SELinux enforces strict system-wide security policies.

In an SELinux-enabled environment, every process, file, directory, network port, and inter-process communication (IPC) channel is labeled with a security context. Even if a user has root privileges, an unauthorized action or a compromised service (like a web server) is strictly restricted to its designated domain and resources.

SELinux Operating Modes

SELinux operates in one of three primary modes:

Enforcing: SELinux policy is strictly enforced. Unauthorized access attempts are blocked and immediately logged as Denial events. This is the default and required operating state for secure production environments.

Permissive: SELinux policy is not enforced, meaning actions are not blocked. However, policy violations are still logged as warnings (AVC denials). This mode is ideal for troubleshooting application issues and developing custom policies without breaking services.

Disabled: SELinux is completely turned off. The kernel does not load any policy, and security contexts are neither enforced nor tracked on the file system.

Important: Never set SELinux to disabled! If an application or service causes security denials, temporarily switch SELinux to Permissive mode instead. Disabling SELinux stops the kernel from applying security labels to newly created or modified files. If SELinux is re-enabled later, the system will require a full file system relabeling process during the next reboot (e.g., via touch /.autorelabel), which can cause significant downtime on large enterprise volumes.

```
# Get the current mode only (Enforcing, Permissive, or Disabled)
getenforce
```

```
dev01@alma-exam:~$ getenforce
Enforcing
dev01@alma-exam:~$
```

```
# Display detailed status information
sestatus
```

```
dev01@alma-exam:~$ sestatus
SELinux status:                 enabled
SELinuxfs mount:                /sys/fs/selinux
SELinux root directory:         /etc/selinux
Loaded policy name:             targeted
Current mode:                   enforcing
Mode from config file:          enforcing
Policy MLS status:              enabled
Policy deny_unknown status:     allowed
Memory protection checking:     actual (secure)
Max kernel policy version:      33
dev01@alma-exam:~$
```

SELinux status (enabled): Indicates whether the SELinux security framework is loaded and active in the kernel.

Loaded policy name (targeted): Defines the active policy type. The default targeted policy restricts specific network services and daemons through process domains and file labeling while allowing unconfined domains to run normally.
Outside the course Scope:
minimum: A lightweight version of targeted that isolates only a tiny set of core base services.
mls (Multi-Level Security): A highly strict policy model. Enforces hierarchical data classifications (e.g., Unclassified to Top Secret). Even root is strictly restricted under MLS.
strict (Legacy): Contains no unconfined processes; every process and user shell must be explicitly permitted by a rule.

Current mode (enforcing): Shows the active operating state in memory at runtime.

Mode from config file (enforcing): Displays the persistent mode configured in /etc/selinux/config that will be applied upon the next system boot.

```
# Set SELinux Mode temporary (0=permissive, 1=enforcing)
sudo setenforce 0
```

```
dev01@alma-exam:~$ sudo setenforce 0
dev01@alma-exam:~$ getenforce
Permissive
dev01@alma-exam:~$
```

```
# Set SELinux Mode persistent in (/etc/selinux/config)
sudo vim /etc/selinux/config
```

```
#
SELINUX=enforcing
# SELINUXTYPE= can take one of these three values:
#     targeted - Targeted processes are protected,
#     minimum - Modification of targeted policy. Only selected processes are protected.
#     mls - Multi Level Security protection.
SELINUXTYPE=targeted
```

SELinux Labels (Contexts)

SELinux labels, also known as security contexts, are security attributes assigned to files, processes, and network ports in the form of user:role:type:level. They are used to enforce mandatory access control policies that go beyond traditional discretionary permissions like rwx, ensuring that processes and users can only access the specific resources explicitly required for their authorized functions. This limits potential security breaches and prevents unauthorized actions, even if a service or user account is compromised.

A well-known example of an SELinux label is the context applied to files served by the Apache web server: system_u:object_r:httpd_sys_content_t:s0

user (system_u): The SELinux user identity, indicating a system service context.
role (object_r): The SELinux role, where object_r is used for static resources like files and directories.
type (httpd_sys_content_t): The domain or type that defines access rules, allowing the web server process to read these files.
level (s0): The Multi-Level Security / Multi-Category Security (MLS/MCS) level

We will go through a few practical examples to make the understanding clear, including working with a file in the home directory and later installing the httpd web server.

```
# Create a file in the home directory
sudo touch example.txt

# Display the SELinux Context
ls -Z example.txt
```

```
dev01@alma-exam:~$ touch example.txt
dev01@alma-exam:~$ ls -Z example.txt
unconfined_u:object_r:user_home_t:s0 example.txt
dev01@alma-exam:~$
```

Field 1, SELinux User (unconfined_u):

The first field represents the SELinux user identity associated with the file. The term "unconfined" indicates that the user or process operates under standard Linux Discretionary Access Control (DAC) permissions with minimal additional SELinux restrictions. In this instance, "unconfined_u" appears because the file was created by the logged-in user account (dev01), whose active session runs under the unconfined SELinux user domain. SELinux automatically inherits the SELinux user context from the process initiating file creation (the shell) and attaches it to the target file metadata.
We will examine SELinux user contexts in greater detail later in this document.

Field 2, SELinux Role (object_r):

The second field specifies the SELinux role. SELinux employs Role-Based Access Control (RBAC) to define permitted system behaviors based on roles. A fundamental distinction exists between Subjects and Objects within the security model:

A Subject is an active entity (such as a running process or user shell) that performs system operations. Subjects operate under functional roles (e.g., unconfined_r, staff_r, or sysadm_r) to control domain transitions and executable permissions.

An Object is a passive entity (such as a file, directory, socket, or system resource) that acts as the target of an operation. Because passive objects cannot execute actions or transition between operational roles, the generic role "object_r" on example.txt serves as a required placeholder to complete the standard SELinux security context format.
We will examine SELinux role contexts in greater detail later in this document.

Field 3, SELinux Type (user_home_t):

The third field defines the SELinux type, which is the core component of Type Enforcement (TE). In the SELinux security model, every object (such as a file, directory, or socket) is assigned a Type, while every active subject runs inside a Domain. Access control decisions depend primarily on rules specifying whether a given domain is permitted to interact with a specific target type (object). In this example, the file received the type "user_home_t" because it was created inside a user home directory. SELinux policy enforces that only appropriate user domains can access files with this type, preventing unauthorized system daemons (such as web servers or database services) from reading personal user files, regardless of traditional Linux permissions.

```
# Change type temporary
chcon -t httpd_sys_content_t example.txt

# Display the SELinux Context
ls -Z example.txt

# Restore Type
restorecon -v example.txt
```

```
dev01@alma-exam:~$ chcon -t httpd_sys_content_t example.txt
dev01@alma-exam:~$ ls -Z example.txt
unconfined_u:object_r:httpd_sys_content_t:s0 example.txt
dev01@alma-exam:~$ restorecon -v example.txt
Relabeled /home/dev01/example.txt from unconfined_u:object_r:httpd_sys_content_t:s0 to unconfined_u:object_r:user_home_t:s0
dev01@alma-exam:~$ ls -Z example.txt
unconfined_u:object_r:user_home_t:s0 example.txt
dev01@alma-exam:~$
```

```
# Move the file to the /tmp directory
mv example.txt /tmp/example.txt

# Change type temporary
chcon -t httpd_sys_content_t example.txt

# Display the SELinux Context
ls -Z /tmp/example.txt

# Restore Type
restorecon -v /tmp/example.txt
```

```
dev01@alma-exam:~$ mv example.txt /tmp/example.txt
dev01@alma-exam:~$ ls -Z /tmp/example.txt
unconfined_u:object_r:user_home_t:s0 /tmp/example.txt
dev01@alma-exam:~$ chcon -t httpd_sys_content_t /tmp/example.txt
dev01@alma-exam:~$ ls -Z /tmp/example.txt
unconfined_u:object_r:httpd_sys_content_t:s0 /tmp/example.txt
dev01@alma-exam:~$ restorecon -v /tmp/example.txt
Warning no default label for /tmp/example.txt
dev01@alma-exam:~$ ls -Z /tmp/example.txt
unconfined_u:object_r:httpd_sys_content_t:s0 /tmp/example.txt
dev01@alma-exam:~$
```

Context Behavior with cp vs. mv Commands:

Moving Files (mv): Executing a standard move operation preserves the original SELinux security context of the file. Because the move operation re-links the existing file inode rather than recreating the file, its metadata including the SELinux extended attributes remains untouched unless moved across different filesystems.

Copying Files (cp): Executing a standard copy operation creates a completely new target file. As a result, the new file does not inherit the security context from the source object, but instead inherits the default SELinux context of the destination directory. To preserve the original SELinux context during a copy operation, the explicit flag --preserve=context must be passed.

Storage Mechanism and Functionality of SELinux Contexts:

SELinux context rules and default path mappings are stored in system policy files located under /etc/selinux/targeted/contexts/files/ (specifically within the file_contexts database and local configuration files). On disk, the active SELinux context is written directly to the file's extended attributes (xattr) on supported filesystems (such as XFS or ext4). When the restorecon command is executed, it reads the absolute file path, compares it against the regular expressions defined in the central file_contexts policy database, determines the expected default context, and overwrites the file's extended attribute accordingly.

As demonstrated in the example above, restorecon was used to reset the type back to the home directory default.

Important Note: chcon only changes the SELinux type temporarily and will be reset after a filesystem relabel or reboot. To change the type permanently, use semanage fcontext -a -t <type> "<file_path_pattern>"

Httpd Examples

To demonstrate and explore advanced SELinux concepts such as type enforcement conflicts, policy booleans, network port security, and troubleshooting techniques, the Apache web server (httpd) is installed and configured as a practical test environment.

Installing httpd:

```
# Install httpd
sudo dnf install httpd -y

# Enabling and starting the httpd deamon
sudo systemctl enable --now httpd

# Checking httpd status
sudo systemctl status httpd
```

```
dev01@alma-exam:~$ sudo dnf install httpd -y
[sudo] password for dev01:
Last metadata expiration check: 3:38:07 ago on Mon 12 Oct 2026 02:39:03 PM CEST.
Package httpd-2.4.63-13.el10_2.6.x86_64 is already installed.
Dependencies resolved.
Nothing to do.
Complete!
dev01@alma-exam:~$ sudo systemctl enable --now httpd
Created symlink '/etc/systemd/system/multi-user.target.wants/httpd.service' → '/usr/lib/systemd/system/httpd.service'.
dev01@alma-exam:~$ sudo systemctl status httpd
● httpd.service - The Apache HTTP Server
     Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
     Active: active (running) since Mon 2026-10-12 18:17:37 CEST; 11s ago
 Invocation: 5c0ebbe5cb914917bfd93acfb58451b7
       Docs: man:httpd.service(8)
   Main PID: 6317 (httpd)
     Status: "Total requests: 0; Idle/Busy workers 100/0;Requests/sec: 0; Bytes served/sec:   0 B/sec"
      Tasks: 177 (limit: 48528)
     Memory: 18.6M (peak: 19M)
        CPU: 98ms
     CGroup: /system.slice/httpd.service
             ├─6317 /usr/sbin/httpd -DFOREGROUND
             ├─6318 /usr/sbin/httpd -DFOREGROUND
             ├─6319 /usr/sbin/httpd -DFOREGROUND
             ├─6320 /usr/sbin/httpd -DFOREGROUND
             └─6322 /usr/sbin/httpd -DFOREGROUND

Oct 12 18:17:35 alma10 systemd[1]: Starting httpd.service - The Apache HTTP Server...
Oct 12 18:17:35 alma10 (httpd)[6317]: httpd.service: Referenced but unset environment variable evaluates to an empty string: OPTIONS
Oct 12 18:17:36 alma10 httpd[6317]: AH00558: httpd: Could not reliably determine the server's fully qualified domain name, using fe80::20c:29>
Oct 12 18:17:37 alma10 httpd[6317]: Server configured, listening on: port 80
Oct 12 18:17:37 alma10 systemd[1]: Started httpd.service - The Apache HTTP Server.
dev01@alma-exam:~$
```

Practical Test: Context Conflict and Access Denial with Apache

```
# Create File in the html directory
sudo touch /var/www/html/test.html
echo "Hello" | sudo tee /var/www/html/test.html

# Display the SELinux Context
ls -Z /var/www/html/test.html

# Change the SELinux type context
sudo chcon -t user_home_t /var/www/html/test.html

# Restore the SELinux type context
sudo restorecon -v /var/www/html/test.html

# Test the Page access
curl http://localhost/test.html
```

```
dev01@alma-exam:~$ sudo touch /var/www/html/test.html
dev01@alma-exam:~$ echo "Hello" | sudo tee /var/www/html/test.html
Hello
dev01@alma-exam:~$ ls -Z /var/www/html/test.html
unconfined_u:object_r:httpd_sys_content_t:s0 /var/www/html/test.html
dev01@alma-exam:~$ curl http://localhost/test.html
Hello
dev01@alma-exam:~$ sudo chcon -t user_home_t /var/www/html/test.html
dev01@alma-exam:~$ ls -Z /var/www/html/test.html
unconfined_u:object_r:user_home_t:s0 /var/www/html/test.html
dev01@alma-exam:~$ curl http://localhost/test.html
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>403 Forbidden</title>
</head><body>
<h1>Forbidden</h1>
<p>You don't have permission to access this resource.</p>
</body></html>
dev01@alma-exam:~$ sudo restorecon -v /var/www/html/test.html
Relabeled /var/www/html/test.html from unconfined_u:object_r:user_home_t:s0 to unconfined_u:object_r:httpd_sys_content_t:s0
dev01@alma-exam:~$ ls -Z /var/www/html/test.html
unconfined_u:object_r:httpd_sys_content_t:s0 /var/www/html/test.html
dev01@alma-exam:~$ curl http://localhost/test.html
Hello
dev01@alma-exam:~$
```

When SELinux evaluates file access requests in Enforcing mode, it performs a Type Enforcement check by comparing the security context of the active process (the subject) against the security context of the target file (the object). Specifically, SELinux checks whether the process domain such as the Apache web server running under the "httpd_t" domain has explicit permission in the policy to access a file assigned to the specific target type (such as "user_home_t"). If the active policy rules do not explicitly permit the process domain to interact with that target object type, SELinux blocks the operation immediately and records an Access Vector Cache (AVC) denial. The underlying binary security policies that define these rules are compiled and loaded into the kernel from storage files located under the /etc/selinux/targeted/policy/ directory.

SELinux Logs and Troubleshooting

SELinux access denials can be inspected manually by examining the primary audit log file located at /var/log/audit/audit.log or by querying the systemd journal using commands like journalctl -g AVC or the specialized utility ausearch. To expand this capability with advanced diagnostic and management tools, administrators can install additional packages using the command sudo dnf install policycoreutils-python-utils setroubleshoot-server -y.

This installation provides a comprehensive administrative toolkit divided into distinct functional areas. The policycoreutils-python-utils package includes utilities such as audit2why, which translates raw, cryptic denial log entries into clear explanations of policy failures, and audit2allow, which can automatically generate custom policy modules when standard configuration adjustments are not sufficient. Furthermore, this package supplies semanage, which manages persistent SELinux configurations such as custom file contexts and network port rules. Meanwhile, the setroubleshoot-server package supplies sealert, a service that analyzes system logs to produce detailed diagnostic reports complete with actionable, step-by-step remediation recommendations.

Certain advanced SELinux management and diagnostic tools are not installed by default on minimal Linux installations such as Red Hat Enterprise Linux, Rocky Linux, or AlmaLinux. Standard deployments only include basic SELinux user-space utilities for checking or modifying execution modes, such as getenforce and setenforce. Consequently, administration packages including policycoreutils-python-utils and setroubleshoot-server must be explicitly installed to enable comprehensive troubleshooting, policy generation, and port management capabilities.

```
# Search in audit.log for httpd avc logs
sudo grep -i avc /var/log/audit/audit.log | grep -i httpd
```

```
dev01@alma-exam:~$ sudo grep -i avc /var/log/audit/audit.log | grep -i httpd
[sudo] password for dev01:
type=AVC msg=audit(1791550670.446:806): avc:  denied  { read } for  pid=5351 comm="httpd" name="verboten.txt" dev="dm-0" ino=68761790 scontext
=system_u:system_r:httpd_t:s0 tcontext=unconfined_u:object_r:user_home_t:s0 tclass=file permissive=0
type=AVC msg=audit(1791822695.600:845): avc:  denied  { read } for  pid=6319 comm="httpd" name="test.html" dev="dm-0" ino=101146313 scontext=s
ystem_u:system_r:httpd_t:s0 tcontext=unconfined_u:object_r:user_home_t:s0 tclass=file permissive=1
type=AVC msg=audit(1791822695.600:845): avc:  denied  { open } for  pid=6319 comm="httpd" path="/var/www/html/test.html" dev="dm-0" ino=101146
313 scontext=system_u:system_r:httpd_t:s0 tcontext=unconfined_u:object_r:user_home_t:s0 tclass=file permissive=1
type=AVC msg=audit(1791822695.600:846): avc:  denied  { map } for  pid=6319 comm="httpd" path="/var/www/html/test.html" dev="dm-0" ino=1011463
13 scontext=system_u:system_r:httpd_t:s0 tcontext=unconfined_u:object_r:user_home_t:s0 tclass=file permissive=1
type=AVC msg=audit(1791822786.785:898): avc:  denied  { read } for  pid=6322 comm="httpd" name="test.html" dev="dm-0" ino=101146313 scontext=s
ystem_u:system_r:httpd_t:s0 tcontext=unconfined_u:object_r:user_home_t:s0 tclass=file permissive=0
type=AVC msg=audit(1791823038.925:1142): avc:  denied  { read } for  pid=6322 comm="httpd" name="test.html" dev="dm-0" ino=101146312 scontext=
system_u:system_r:httpd_t:s0 tcontext=unconfined_u:object_r:user_home_t:s0 tclass=file permissive=0
dev01@alma-exam:~$ sudo ausearch -m AVC -ts today -i
```

```
#Show SELinux Logs with ausearch
sudo ausearch -m AVC -ts today -i
```

```
dev01@alma-exam:~$ sudo ausearch -m AVC -ts today -i
----
type=PROCTITLE msg=audit(10/12/2026 18:31:35.600:845) : proctitle=/usr/sbin/httpd -DFOREGROUND
type=SYSCALL msg=audit(10/12/2026 18:31:35.600:845) : arch=x86_64 syscall=openat success=yes exit=14 a0=AT_FDCWD a1=0x7f0a4c004938 a2=O_RDONLY
|O_CLOEXEC a3=0x0 items=0 ppid=6317 pid=6319 auid=unset uid=apache gid=apache euid=apache suid=apache fsuid=apache egid=apache sgid=apache fsg
id=apache tty=(none) ses=unset comm=httpd exe=/usr/sbin/httpd subj=system_u:system_r:httpd_t:s0 key=(null)
type=AVC msg=audit(10/12/2026 18:31:35.600:845) : avc:  denied  { open } for  pid=6319 comm=httpd path=/var/www/html/test.html dev="dm-0" ino=
101146313 scontext=system_u:system_r:httpd_t:s0 tcontext=unconfined_u:object_r:user_home_t:s0 tclass=file permissive=1
type=AVC msg=audit(10/12/2026 18:31:35.600:845) : avc:  denied  { read } for  pid=6319 comm=httpd name=test.html dev="dm-0" ino=101146313 scon
text=system_u:system_r:httpd_t:s0 tcontext=unconfined_u:object_r:user_home_t:s0 tclass=file permissive=1
----
```

Pipe Ausearch Message into audit2why

```
sudo ausearch -m avc -ts today -i | audit2why
```

```
dev01@alma-exam:~$ sudo ausearch -m avc -ts today -i | audit2why
[sudo] password for dev01:
type=AVC msg=audit(10/12/2026 18:31:35.600:845) : avc:  denied  { open } for  pid=6319 comm=httpd path=/var/www/html/test.html dev="dm-0" ino=
101146313 scontext=system_u:system_r:httpd_t:s0 tcontext=unconfined_u:object_r:user_home_t:s0 tclass=file permissive=1

        Was caused by:
        The boolean httpd_read_user_content was set incorrectly.
        Description:
        Allow httpd to read user content

        Allow access by executing:
        # setsebool -P httpd_read_user_content 1
type=AVC msg=audit(10/12/2026 18:31:35.600:845) : avc:  denied  { read } for  pid=6319 comm=httpd name=test.html dev="dm-0" ino=101146313 scon
text=system_u:system_r:httpd_t:s0 tcontext=unconfined_u:object_r:user_home_t:s0 tclass=file permissive=1

        Was caused by:
        The boolean httpd_read_user_content was set incorrectly.
        Description:
        Allow httpd to read user content

        Allow access by executing:
        # setsebool -P httpd_read_user_content 1
```

Create a full report with sealert

```
sudo sealert -a /var/log/audit/audit.log
```

```
dev01@alma-exam:~$ sudo sealert -a /var/log/audit/audit.log
100% done
found 3 alerts in /var/log/audit/audit.log
--------------------------------------------------------------------------------

SELinux is preventing /usr/sbin/httpd from read access on the file test.html.

*****  Plugin restorecon (92.2 confidence) suggests   ************************

If you want to fix the label.
test.html default label should be httpd_sys_content_t.
Then you can run restorecon. The access attempt may have been stopped due to insufficient permissions to access a parent directory in which ca
se try to change the following command accordingly.
Do
# /sbin/restorecon -v test.html

*****  Plugin catchall_boolean (7.83 confidence) suggests   ******************

If you want to allow httpd to read user content
Then you must tell SELinux about this by enabling the 'httpd_read_user_content' boolean.
```

As seen in the previous output, the context of the HTML file did not match because the file type was set to user_home_t instead of the expected httpd_t, which triggered the access denied error resulting in a 403 status code. Tools like audit2why and sealert serve as great helpers to inspect not only the logs but also receive actionable recommendations. Let us implement the recommendation provided by audit2why and immediately move on to the next topic, SELinux booleans, keeping in mind that this is merely an example and does not imply that such changes should be implemented this way in a production environment, as this serves strictly for testing purposes.

SELinux Booleans

SELinux Booleans are configuration switches that allow parts of the SELinux policy to be changed at runtime without requiring custom policy compilation. They act as on/off switches that enable administrators to flexibly adjust security rules, such as allowing a web server to make network connections.

They are not stored in the same location as the core policy files (which reside under /etc/selinux/{SELINUXTYPE}/policy/). Instead, active booleans are managed in-memory and exposed through a pseudo-filesystem at /sys/fs/selinux/booleans/, while persistent changes are stored separately in /etc/selinux/{SELINUXTYPE}/modules/active/booleans.local.

```
# Show SELinux Booleans in the current runtime
getsebool -a
```

```
dev01@alma-exam:~$ getsebool -a
abrt_anon_write --> off
abrt_handle_event --> on
```

```
# Show more Info for Booleans, description, current runtime and persistent setting
sudo semanage boolean -l
```

```
dev01@alma-exam:~$ sudo semanage boolean -l
[sudo] password for dev01:
SELinux boolean                State  Default Description

abrt_anon_write                (off  ,  off)  Allow abrt to anon write
abrt_handle_event              (on   ,   on)  Allow abrt to handle event
```

```
# Set Boolean temporary to on with keyword on or 1
sudo setsebool httpd_enable_homedirs on
sudo setsebool httpd_enable_homedirs 1

# Set Boolean temporary to off with keyword off or 0
sudo setsebool httpd_enable_homedirs off
sudo setsebool httpd_enable_homedirs 0

# Set Boolean to off or on persistent
sudo setsebool -P httpd_enable_homedirs off
sudo semanage boolean --modify --on httpd_enable_homedirs
```

```
dev01@alma-exam:~$ sudo semanage boolean -l | grep httpd
httpd_anon_write               (off  ,  off)  Allow httpd to anon write
httpd_builtin_scripting        (on   ,   on)  Allow httpd to builtin scripting
httpd_can_check_spam           (off  ,  off)  Allow httpd to can check spam
httpd_can_connect_ftp          (off  ,  off)  Allow httpd to can connect ftp
httpd_can_connect_ldap         (off  ,  off)  Allow httpd to can connect ldap
httpd_can_connect_mythtv       (off  ,  off)  Allow httpd to can connect mythtv
httpd_can_connect_zabbix       (off  ,  off)  Allow httpd to can connect zabbix
httpd_can_manage_courier_spool (off  ,  off)  Allow httpd to can manage courier spool
httpd_can_network_connect      (off  ,  off)  Allow httpd to can network connect
httpd_can_network_connect_cobbler (off  ,  off)  Allow httpd to can network connect cobbler
httpd_can_network_connect_db   (off  ,  off)  Allow httpd to can network connect db
httpd_can_network_memcache     (off  ,  off)  Allow httpd to can network memcache
httpd_can_network_redis        (off  ,  off)  Allow httpd to can network redis
httpd_can_network_relay        (off  ,  off)  Allow httpd to can network relay
httpd_can_sendmail             (off  ,  off)  Allow httpd to can sendmail
httpd_dbus_avahi               (off  ,  off)  Allow httpd to dbus avahi
httpd_dbus_sssd                (off  ,  off)  Allow httpd to dbus sssd
httpd_dontaudit_search_dirs    (off  ,  off)  Allow httpd to dontaudit search dirs
httpd_enable_cgi               (on   ,   on)  Allow httpd to enable cgi
httpd_enable_ftp_server        (off  ,  off)  Allow httpd to enable ftp server
httpd_enable_homedirs          (on   ,  off)  Allow httpd to enable homedirs
```

After enabling the httpd_enable_homedirs boolean, our curl access is now working again.

SELinux Port Contexts

SELinux port contexts are an integral component of SELinux policies that define the security labels assigned to network ports. They act as a mapping mechanism between network ports (such as TCP or UDP ports) and SELinux types, enabling the policy to enforce granular access control over network services. By integrating port contexts into the security policy, SELinux ensures that processes running in specific domains can only bind to or communicate over designated network ports.

```
# Set LISTEN port to 82 in httpd.conf
sudo vim /etc/httpd/conf/httpd.conf

# Check which Ports are allowed for httpd SELinux and firewall-cmd
sudo semanage port -l | grep http
sudo firewall-cmd --list-ports

# Add Ports in SELinux and firewall-cmd
sudo semanage port -a -t http_port_t -p tcp 82
sudo firewall-cmd --permanent --add-port=82/tcp

# Restart the httpd deamon
sudo systemctl restart httpd
```

```
dev01@alma10:~$ sudo vim /etc/httpd/conf/httpd.conf
dev01@alma10:~$ sudo systemctl restart httpd
Job for httpd.service failed because the control process exited with error code.
See "systemctl status httpd.service" and "journalctl -xeu httpd.service" for details.
dev01@alma10:~$ sudo semanage port -l | grep http
http_cache_port_t              tcp      8080, 8118, 8123, 10001-10010
http_cache_port_t              udp      3130
http_port_t                    tcp      8181, 80, 81, 443, 488, 8008, 8009, 8443, 9000
http_port_t                    udp      80, 443
pegasus_http_port_t            tcp      5988
pegasus_https_port_t           tcp      5989
dev01@alma10:~$ sudo firewall-cmd --list-ports
8181/tcp
dev01@alma10:~$ sudo semanage port -a -t http_port_t -p tcp 82


dev01@alma10:~$ sudo semanage port -l | grep http
http_cache_port_t              tcp      8080, 8118, 8123, 10001-10010
http_cache_port_t              udp      3130
http_port_t                    tcp      82, 8181, 80, 81, 443, 488, 8008, 8009, 8443, 9000
http_port_t                    udp      80, 443
pegasus_http_port_t            tcp      5988
pegasus_https_port_t           tcp      5989
dev01@alma10:~$ sudo firewall-cmd --list-ports
8181/tcp
dev01@alma10:~$ sudo firewall-cmd --permanent --add-port=82/tcp
success
dev01@alma10:~$ sudo firewall-cmd --list-ports
8181/tcp
dev01@alma10:~$ sudo firewall-cmd --reload
success
dev01@alma10:~$ sudo firewall-cmd --list-ports
82/tcp 8181/tcp
dev01@alma10:~$ sudo systemctl restart httpd
dev01@alma10:~$
```

SELinux Architecture: SELinux User and Roles (RBAC)

The SELinux Security Context In SELinux, every subject (active process) and every object (passive resource such as files, sockets, or directories) is identified by a security context (label). This context follows a standard structure:

SELinux-User : Role : Type : Security-Level

Example Process: staff_u : staff_r : staff_t : s0 Example File: system_u : object_r : httpd_sys_content_t : s0

SELinux User An SELinux user is an entity distinct from traditional Linux user accounts (/etc/passwd). Linux users are mapped to SELinux users through assignment tables (viewable via sudo semanage login -l).

For Subjects (Processes): The SELinux user defines the top-level identity boundary. It specifies which SELinux roles a user or process is permitted to assume.

For Objects (Files): Files typically carry the SELinux user of their creator or a system default ('system_u'). For objects, the SELinux user serves primarily for tracking origin and plays a minor role in daily access control decisions.

SELinux Role (RBAC) Roles form the core of Role-Based Access Control (RBAC) in SELinux. By convention, role names end with the suffix '_r'.

For Subjects (Processes): A role acts as a bridge between the SELinux user and executable types (domains). The role determines which process types (domains) a subject can transition into and which operations are authorized within that role.

For Objects (Files): Because static files do not execute actions, filesystem objects almost exclusively use the generic placeholder role 'object_r'.

Interaction of RBAC and Type Enforcement (TE) The SELinux security model combines RBAC and Type Enforcement into layered controls:

User to Role: Controls which roles can be assigned to a given SELinux user.

Role to Type (RBAC): Controls which types/domains a given role can assume or activate.

Type to Type (Type Enforcement): Controls the specific permissions a process type (domain) has over an object type (file, port, etc.).

Comparison: Subject vs. Object

SELinux User

Subject (Process): Defines the authorized roles for the process.

Object (File/Resource): Documents the creator or origin of the object (typically 'system_u').

SELinux Role

Subject (Process): Defines the authorized process domains (RBAC).

Object (File/Resource): Uses the universal placeholder 'object_r'.

SELinux Type

Subject (Process): Defines the process domain (e.g., 'httpd_t').

Object (File/Resource): Defines the file type (e.g., 'httpd_sys_content_t').

Policy Rules: Allow and Deny

SELinux operates on a strict default-deny principle. Any access attempt that is not explicitly authorized by a rule is blocked by default.

Allowed Access (Allow Rule) An explicit rule grants a domain permission to access a specific file type:

allow httpd_t httpd_sys_content_t:file { read getattr open };

- Subject: 'httpd_t' (process domain of the web server)
- Object: 'httpd_sys_content_t' (file type for web content)
- Object Class: 'file' (filesystem object class)
- Permissions: Read access, getting attributes, and opening the file.

For this access to occur, the subject must have previously been authorized to assume the 'system_r' role, which permits launching the 'httpd_t' domain.

Denied Access (Deny Scenarios)

Implicit Deny (Missing Rule): If the process 'httpd_t' attempts to read a file labeled 'shadow_t' (/etc/shadow), no 'allow' rule exists for this interaction. The access is denied by the kernel, generating an Access Vector Cache (AVC) audit entry:

type=AVC msg=audit(1776060000.123:456): avc: denied { read } for pid=8821 comm="httpd" path="/etc/shadow" dev="sda1" ino=12345 scontext=system_u:system_r:httpd_t:s0 tcontext=system_u:object_r:shadow_t:s0 tclass=file

4. USBGuard

This guide assumes you are already familiar with the fundamentals of Ansible and have a working configuration established. For the following labs and scenarios, we use a three-node AlmaLinux 10 environment: alma10 serves as the Ansible Control Node, while almaserver01 and almaserver02 act as the Managed Nodes. Ensure that passwordless SSH key-based authentication is fully configured from the control node to both managed nodes before proceeding with automated hardening tasks.

USBGuard is a software framework designed to protect Linux systems against rogue USB devices by implementing basic allow- and block-policy enforcement based on device attributes. It monitors the USB device bus and can automatically authorize or block newly connected devices according to a defined security policy, preventing potential BadUSB attacks, unauthorized data exfiltration, or malicious hardware keyloggers while allowing trusted peripherals like standard keyboards, mice, or company-approved storage media to function normally.

```
# Check if Ansible connections to the control nodes work
ansible all -m ping

# Install USBGuard
sudo dnf install usbguard -y
```

```
dev01@alma10:~/Documents/ansible_test$ ansible all -m ping
192.168.159.129 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "changed": false,
    "ping": "pong"
}
192.168.159.128 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "changed": false,
    "ping": "pong"
}
dev01@alma10:~/Documents/ansible_test$ sudo dnf install usbguard -y
```

```
# Create initial policy with already connected usb devices
sudo usbguard generate-policy | sudo tee /etc/usbguard/rules.conf

# Start USBGuard daemon
sudo systemctl enable --now usbguard
```

```
dev01@alma10:~/Documents/ansible_test$ sudo usbguard generate-policy | sudo tee /etc/usbguard/rules.conf
allow id 1d6b:0002 serial "0000:02:02.0" name "EHCI Host Controller" hash "8G9+DxPfLbVnCuw59P+Fxfu6Yxy0AVsuFJ3qRPS0XNM=" parent-hash "Ps1yU1JiZl/dOj3LXAXp9uW
QiNXeY3IT3NGW0E96O30=" with-interface 09:00:00 with-connect-type ""
allow id 1d6b:0001 serial "0000:02:00.0" name "UHCI Host Controller" hash "3QuFY0nRVqo+mgnIkqiXmmirRvxPDeP7P3mintodpys=" parent-hash "42PXRxhXx8adD0H10oXgQnn
a1ED6vikevkSl6cwgdOI=" with-interface 09:00:00 with-connect-type ""
allow id 1d6b:0002 serial "0000:0b:00.0" name "xHCI Host Controller" hash "+XquLJ1PcxL46fLrSbVNwpIn3oJQYx8VHJUWeqlOsws=" parent-hash "7dcv/cFxYahX1McsTHTTEXF
D44eU2mZn3FtbwRrYmUU=" with-interface 09:00:00 with-connect-type ""
allow id 1d6b:0003 serial "0000:0b:00.0" name "xHCI Host Controller" hash "J4JlQX4Hwd5BzruL3wHVfm9e9H5qBBEkonWzdhKjNXc=" parent-hash "7dcv/cFxYahX1McsTHTTEXF
D44eU2mZn3FtbwRrYmUU=" with-interface 09:00:00 with-connect-type ""
allow id 0e0f:0003 serial "" name "VMware Virtual USB Mouse" hash "F4BFfAiFQzjQxG8oQHVYMFiDF2/zhhARDrRSIj6oij8=" parent-hash "+XquLJ1PcxL46fLrSbVNwpIn3oJQYx8
```

```
# Verify added devices
sudo usbguard list-devices
```

To configure USBGuard properly, **the initial policy must be generated before starting the daemon.** This ensures that all currently connected devices and system controllers are recorded and allowed. It is critical to perform this step first, because launching the daemon with an empty policy file while the default block rule is active can cause administrative lockouts.

```
# Verify blocked usb stick after plugin
sudo usbguard list-devices

# Allow plugged in usb stick temporary
sudo usbguard allow-device 11

# Allow plugged in usb stick permanent
sudo usbguard allow-device -p 11
```

```
dev01@alma10:~/Documents/ansible_test$ sudo usbguard list-devices
[sudo] password for dev01:
6: allow id 1d6b:0002 serial "0000:02:02.0" name "EHCI Host Controller" hash "8G9+DxPfLbVnCuw59P+Fxfu6Yxy0AVsuFJ3qRPS0XNM=" parent-hash "Ps1yU1JiZl/dOj3LXAXp9uWQiNXeY3IT3NGW0E96O30=" via-port "usb1" with-interface 09:00:00 with-connect-type ""
7: allow id 1d6b:0001 serial "0000:02:00.0" name "UHCI Host Controller" hash "3QuFY0nRVqo+mgnIkqiXmmirRvxPDeP7P3mintodpys=" parent-hash "42PXRxhXx8adD0H10oXgQnna1ED6vikevkSl6cwgdOI=" via-port "usb2" with-interface 09:00:00 with-connect-type ""
8: allow id 1d6b:0002 serial "0000:0b:00.0" name "xHCI Host Controller" hash "+XquLJ1PcxL46fLrSbVNwpIn3oJQYx8VHJUWeqlOsws=" parent-hash "7dcv/cFxYahX1McsTHTTEXFD44eU2mZn3FtbwRrYmUU=" via-port "usb3" with-interface 09:00:00 with-connect-type ""
9: allow id 1d6b:0003 serial "0000:0b:00.0" name "xHCI Host Controller" hash "J4JlQX4Hwd5BzruL3wHVfm9e9H5qBBEkonWzdhKjNXc=" parent-hash "7dcv/cFxYahX1McsTHTTEXFD44eU2mZn3FtbwRrYmUU=" via-port "usb4" with-interface 09:00:00 with-connect-type ""
10: allow id 0e0f:0003 serial "" name "VMware Virtual USB Mouse" hash "F4BFfAiFQzjQxG8oQHVYMFiDF2/zhhARDrRSIj6oij8=" parent-hash "+XquLJ1PcxL46fLrSbVNwpIn3oJQYx8VHJUWeqlOsws=" via-port "3-1" with-interface 03:01:02 with-connect-type "unknown"
dev01@alma10:~/Documents/ansible_test$ sudo usbguard list-devices
6: allow id 1d6b:0002 serial "0000:02:02.0" name "EHCI Host Controller" hash "8G9+DxPfLbVnCuw59P+Fxfu6Yxy0AVsuFJ3qRPS0XNM=" parent-hash "Ps1yU1JiZl/dOj3LXAXp9uWQiNXeY3IT3NGW0E96O30=" via-port "usb1" with-interface 09:00:00 with-connect-type ""
7: allow id 1d6b:0001 serial "0000:02:00.0" name "UHCI Host Controller" hash "3QuFY0nRVqo+mgnIkqiXmmirRvxPDeP7P3mintodpys=" parent-hash "42PXRxhXx8adD0H10oXgQnna1ED6vikevkSl6cwgdOI=" via-port "usb2" with-interface 09:00:00 with-connect-type ""
8: allow id 1d6b:0002 serial "0000:0b:00.0" name "xHCI Host Controller" hash "+XquLJ1PcxL46fLrSbVNwpIn3oJQYx8VHJUWeqlOsws=" parent-hash "7dcv/cFxYahX1McsTHTTEXFD44eU2mZn3FtbwRrYmUU=" via-port "usb3" with-interface 09:00:00 with-connect-type ""
9: allow id 1d6b:0003 serial "0000:0b:00.0" name "xHCI Host Controller" hash "J4JlQX4Hwd5BzruL3wHVfm9e9H5qBBEkonWzdhKjNXc=" parent-hash "7dcv/cFxYahX1McsTHTTEXFD44eU2mZn3FtbwRrYmUU=" via-port "usb4" with-interface 09:00:00 with-connect-type ""
10: allow id 0e0f:0003 serial "" name "VMware Virtual USB Mouse" hash "F4BFfAiFQzjQxG8oQHVYMFiDF2/zhhARDrRSIj6oij8=" parent-hash "+XquLJ1PcxL46fLrSbVNwpIn3oJQYx8VHJUWeqlOsws=" via-port "3-1" with-interface 03:01:02 with-connect-type "unknown"
11: block id 058f:6387 serial "9CB9CCD6" name "Mass Storage" hash "mzMqrB6itM/LqgCe1xO0UhTFCzyt5vivv2CFWcKAjSg=" parent-hash "+XquLJ1PcxL46fLrSbVNwpIn3oJQYx8VHJUWeqlOsws=" via-port "3-2" with-interface 08:06:50 with-connect-type "unknown"
12: block id 0e0f:0002 serial "" name "VMware Virtual USB Hub" hash "0cDIzeFX6eYg1fPydeh+aK2a/vRVjjj9bIasbHuN5sE=" parent-hash "+XquLJ1PcxL46fLrSbVNwpIn3oJQYx8VHJUWeqlOsws=" via-port "3-3" with-interface 09:00:00 with-connect-type "unknown"
13: block id 0e0f:0002 serial "" name "VMware Virtual USB Hub" hash "5EH6pwEeKjiAo3isQ2bIHpQTDaN+2blPoJtEknkL8cY=" parent-hash "+XquLJ1PcxL46fLrSbVNwpIn3oJQYx8VHJUWeqlOsws=" via-port "3-4" with-interface 09:00:00 with-connect-type "unknown"
dev01@alma10:~/Documents/ansible_test$ sudo usbguard allow-device -p 11
dev01@alma10:~/Documents/ansible_test$ sudo usbguard list-devices
[sudo] password for dev01:
6: allow id 1d6b:0002 serial "0000:02:02.0" name "EHCI Host Controller" hash "8G9+DxPfLbVnCuw59P+Fxfu6Yxy0AVsuFJ3qRPS0XNM=" parent-hash "Ps1yU1JiZl/dOj3LXAXp9uWQiNXeY3IT3NGW0E96O30=" via-port "usb1" with-interface 09:00:00 with-connect-type ""
7: allow id 1d6b:0001 serial "0000:02:00.0" name "UHCI Host Controller" hash "3QuFY0nRVqo+mgnIkqiXmmirRvxPDeP7P3mintodpys=" parent-hash "42PXRxhXx8adD0H10oXgQnna1ED6vikevkSl6cwgdOI=" via-port "usb2" with-interface 09:00:00 with-connect-type ""
8: allow id 1d6b:0002 serial "0000:0b:00.0" name "xHCI Host Controller" hash "+XquLJ1PcxL46fLrSbVNwpIn3oJQYx8VHJUWeqlOsws=" parent-hash "7dcv/cFxYahX1McsTHTTEXFD44eU2mZn3FtbwRrYmUU=" via-port "usb3" with-interface 09:00:00 with-connect-type ""
9: allow id 1d6b:0003 serial "0000:0b:00.0" name "xHCI Host Controller" hash "J4JlQX4Hwd5BzruL3wHVfm9e9H5qBBEkonWzdhKjNXc=" parent-hash "7dcv/cFxYahX1McsTHTTEXFD44eU2mZn3FtbwRrYmUU=" via-port "usb4" with-interface 09:00:00 with-connect-type ""
10: allow id 0e0f:0003 serial "" name "VMware Virtual USB Mouse" hash "F4BFfAiFQzjQxG8oQHVYMFiDF2/zhhARDrRSIj6oij8=" parent-hash "+XquLJ1PcxL46fLrSbVNwpIn3oJQYx8VHJUWeqlOsws=" via-port "3-1" with-interface 03:01:02 with-connect-type "unknown"
11: allow id 058f:6387 serial "9CB9CCD6" name "Mass Storage" hash "mzMqrB6itM/LqgCe1xO0UhTFCzyt5vivv2CFWcKAjSg=" parent-hash "+XquLJ1PcxL46fLrSbVNwpIn3oJQYx8VHJUWeqlOsws=" via-port "3-2" with-interface 08:06:50 with-connect-type "unknown"
12: block id 0e0f:0002 serial "" name "VMware Virtual USB Hub" hash "0cDIzeFX6eYg1fPydeh+aK2a/vRVjjj9bIasbHuN5sE=" parent-hash "+XquLJ1PcxL46fLrSbVNwpIn3oJQYx8VHJUWeqlOsws=" via-port "3-3" with-interface 09:00:00 with-connect-type "unknown"
13: block id 0e0f:0002 serial "" name "VMware Virtual USB Hub" hash "5EH6pwEeKjiAo3isQ2bIHpQTDaN+2blPoJtEknkL8cY=" parent-hash "+XquLJ1PcxL46fLrSbVNwpIn3oJQYx8VHJUWeqlOsws=" via-port "3-4" with-interface 09:00:00 with-connect-type "unknown"
dev01@alma10:~/Documents/ansible_test$
```

When a standard USB flash drive was plugged into the system, it was immediately blocked by USBGuard, preventing any access to the device. After explicitly allowing the USB stick using the persistent allow command, we verified via the device list that the status changed from blocked to allowed, restoring full access to the device.

USBGuard Rule-Set Documentation

USBGuard rules strictly follow a top-to-bottom processing model (First-Match-Wins). Each rule consists of an action, conditions, and attributes.

Rule Action Types

allow: Authorizes the device and allows data transmission across the USB bus.

block: Keeps the device connected to the USB bus, but prevents the OS from loading drivers or interacting with it.

reject: Blocks the device and forcibly disconnects it from the USB bus at the port level.

Common Match Attributes

id <vendor_id>:<product_id>: Matches exact Vendor and Product hex codes (e.g., id 046d:c52b).with-interface <class>:<subclass>:<protocol>: Matches USB interface classes according to USB-IF specifications.

serial "<serial_number>": Matches the device hardware serial number.

hash "<base64_hash>": Matches the unique cryptographic hash generated by USBGuard for a specific device.

via-port "<port_id>": Binds the rule to a specific physical USB port (e.g., via-port "3-1").

Policy Configuration Scenarios

To allow all devices manufactured by a trusted vendor (e.g., Logitech, Vendor ID 046d) while keeping default blocking for everything else:

Allow all USB devices manufactured by Logitech (Vendor ID: 046d)

allow id 046d:*

Allow Linux Kernel Root Hubs (Required for USB subsystem operation)

allow id 1d6b:*

Allow ALL HID Devices (Keyboards/Mice) while Blocking ALL Storage
To allow input devices regardless of manufacturer, but strictly block USB sticks, external SSDs, and card readers:

Allow Linux Kernel Root Controllers
allow id 1d6b:* with-interface 09:00:00

Allow ALL Human Interface Devices (Class 03)
Matches any HID device (Keyboards, Mice, Barcode Scanners, Gamepads)
allow with-interface 03:*:*

Explicitly BLOCK ALL Mass Storage Devices (Class 08)
Wildcards match all subclasses (e.g., SCSI, RBC) and protocols (e.g., BBB, UASP)
block with-interface 08:*:*

BadUSB / Rubber Ducky Mitigation & Defense Limits
A Rubber Ducky is a hardware attack tool disguised as a standard USB drive that identifies itself to the operating system as a USB keyboard to inject keystrokes (commands).

Reject any single device that presents BOTH a Mass Storage interface and a HID interface
reject with-interface all-of { 08:*:* 03:*:* }

Note:
If a Rubber Ducky is flashed to present only a single HID interface (03:01:01) and spoofs the exact Vendor ID and Product ID of a trusted keyboard on your whitelist, USBGuard will treat it as a legitimate keyboard and allow it.

See more informations for best practices:
https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/8/html/security_hardening/protecting-systems-against-intrusive-usb-devices_security-hardening

Ansible USBGuard Deployment

```
# Deploy usbguard
ansible-playbook -i inventory usbguard_deploy.yml --limit 192.168.159.128
```

```
dev01@alma10:~/Documents/ansible_test$ ansible-playbook -i inventory usbguard_deploy.yml --limit 192.168.159.128

PLAY [Deploy USBGuard from Control Node Policy to Target] **********************************************************************************************************************************************************

TASK [Gathering Facts] *********************************************************************************************************************************************************************************************
ok: [192.168.159.128]

TASK [1. Ensure USBGuard package is installed] *********************************************************************************************************************************************************************
ok: [192.168.159.128]

TASK [2. Copy rules.conf from project directory to managed node] ***************************************************************************************************************************************************
changed: [192.168.159.128]

TASK [3. Ensure usbguard daemon is enabled and started] ************************************************************************************************************************************************************
changed: [192.168.159.128]

PLAY RECAP *********************************************************************************************************************************************************************************************************
192.168.159.128            : ok=4    changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

dev01@alma10:~/Documents/ansible_test$
```

usbguard_deploy.yml

```
# Deployment yml file
---
- name: Deploy USBGuard from Control Node Policy to Target
  hosts: 192.168.159.128
  become: true

  tasks:
    - name: 1. Ensure USBGuard package is installed
      ansible.builtin.dnf:
        name: usbguard
        state: present

    - name: 2. Copy rules.conf from project directory to managed node
      ansible.builtin.copy:
        src: rules.conf
        dest: /etc/usbguard/rules.conf
        owner: root
        group: root
        mode: '0600'

    - name: 3. Ensure usbguard daemon is enabled and started
      ansible.builtin.service:
        name: usbguard
        state: started
        enabled: true
```

5. PAM Basics

PAM stands for Pluggable Authentication Modules which is a system of libraries that integrate multiple low-level authentication methods into a high-level API allowing system administrators to set and change authentication policies without having to rewrite applications. What PAM can do includes controlling how users log in by verifying credentials against local files or external directories like LDAP or SSSD, enforcing account policies such as checking whether an account has expired or if login is allowed at a specific time, managing password complexity requirements to ensure users choose secure passwords, keeping track of failed login attempts to lock out accounts and protect against brute-force attacks, and finally managing sessions by setting up resource limits and executing necessary tasks when a user logs in or out.

PAM configuration files use control flags on each module line to dictate how the system evaluates the success or failure of that specific module during an authentication attempt. The available control flags and their specific functions are as follows:

The requisite flag means the module must succeed for authentication to proceed, and if it fails, the authentication process is aborted immediately without checking any subsequent modules.

The required flag also mandates that the module must succeed for overall access to be granted, but if it fails, PAM continues processing the remaining modules in the stack before ultimately denying access at the end to prevent attackers from discovering which specific rule failed.

The sufficient flag dictates that if the module succeeds, authentication is granted immediately, bypassing all remaining modules in the stack, provided that no previous required module has already failed.

The optional flag means the module's success or failure is generally ignored in determining the final authentication result, as it is typically used for supplementary background actions like logging or environment setup.

pwquality

```
# Show current
authselect current

# Show current
authselect list

# Show current
sudo vim /etc/security/pwquality.conf
```

```
# Minimum acceptable size for the new password (plus one if
# credits are not disabled which is the default). (See pam_cracklib manual.)
# Cannot be set to lower value than 6.
 minlen = 10
#
# The maximum credit for having digits in the new password. If less than 0
# it is the minimum number of digits in the new password.
 dcredit = -1
#
# The maximum credit for having uppercase characters in the new password.
# If less than 0 it is the minimum number of uppercase characters in the new
# password.
 ucredit = -1
#
# The maximum credit for having lowercase characters in the new password.
# If less than 0 it is the minimum number of lowercase characters in the new
# password.
# lcredit = 0
#
# The maximum credit for having other characters in the new password.
# If less than 0 it is the minimum number of other characters in the new
# password.
 ocredit = -1
#
```

```
dev01@almaserver01:~

dev01@almaserver01:~$ sudo useradd testuser
dev01@almaserver01:~$ sudo passwd testuser
New password:
BAD PASSWORD: The password contains less than 1 digits
Retype new password:
Sorry, passwords do not match.
passwd: Authentication token manipulation error
passwd: password unchanged
dev01@almaserver01:~$
```

The available profiles can be viewed using the command **authselect list**.

The local profile configures the system to handle users and authentication exclusively through standard local files such as /etc/passwd and /etc/shadow, making it ideal for standalone servers that do not rely on central directory services.

The sssd profile enables the System Security Services Daemon as the primary authentication backend, which is the modern standard for connecting the system to centralized identity providers like LDAP, FreeIPA, or Active Directory.

The winbind profile integrates the Samba winbind service into the authentication stack, which is specifically used when a Linux server needs to authenticate users directly against a Windows Active Directory domain.

In the **/etc/security/pwquality.conf** file, specific parameters are applied to enforce password complexity rules.

The minlen parameter defines the minimum allowable length for a new password.

The dcredit parameter specifies the minimum number of digits required in the password when set to a negative value.

The ucredit parameter mandates the minimum number of uppercase letters required when configured with a negative integer.

The ocredit parameter requires a minimum number of other characters, such as special symbols, when given a negative value.

When a user attempts to create a new password that fails to meet these defined requirements using the passwd command, the PAM framework intercepts the change request through the pam_pwquality module and immediately rejects it, displaying an error message explaining that the password fails the quality checks or is too short.

Faillock

```
# Change faillock configuration
sudo vim /etc/security/faillock.conf

# Reset faillock for a specific user
sudo faillock --user <username> --reset
```

```
#
# Deny access if the number of consecutive authentication failures
# for this user during the recent interval exceeds n tries.
# The default is 3.
 deny = 3
#
# The length of the interval during which the consecutive
# authentication failures must happen for the user account
# lock out is <replaceable>n</replaceable> seconds.
# The default is 900 (15 minutes).
 fail_interval = 900
#
# The access will be re-enabled after n seconds after the lock out.
# The value 0 has the same meaning as value `never` - the access
# will not be re-enabled without resetting the faillock
# entries by the `faillock` command.
# The default is 600 (10 minutes).
 unlock_time = 600
```

The fail_interval parameter defines the time window, in seconds, during which consecutive authentication failures must occur for the account to become locked out.

The deny parameter specifies the maximum number of allowed authentication failures before the user account is locked. The unlock_time parameter determines how long the account remains locked in seconds before access is automatically re-enabled, with a value of zero meaning the account stays locked indefinitely until manually cleared by an administrator.

The even_deny_root parameter ensures that the root user account is subjected to the exact same lockout rules as regular users upon consecutive failed login attempts.

You got the idea! PAM allows you to fine-tune virtually every aspect of user authentication, session management, and account access using modular controls like pam_pwquality, pam_faillock, pam_pwhistory, and feature flags like with-mkhomedir.

Beyond these core modules, PAM provides many additional capabilities for enterprise hardening:

pam_time / pam_access: Restricts logins based on access rules, networks, origin hosts, or specific time/day windows.

pam_wheel: Restricts administrative commands like su strictly to members of the wheel group.

pam_limits: Sets system resource limits (such as max logins, open files, memory usage) upon session opening.

pam_env: Sets or unsets environment variables for logged-in sessions.

6. Auditd

The Linux Audit framework (auditd) is a kernel-level security auditing system designed to track and record safety-relevant events across your operating system. At its core, auditd runs as a background daemon (system service) that receives event logs directly from the Linux kernel. Because it captures events at the kernel system call level, it reliably logs activity even if user-space applications or privileges are compromised.

Key capabilities of auditd include:

Real-time file system watching to monitor read, write, execution, and attribute modifications on critical directories and files.

Granular system call (syscall) tracking to log process executions, network changes, or unauthorized access attempts.

Comprehensive user context tracking that maps actions back to the original login session user (auid), preserving accountability even across privilege escalation utilities like sudo or su.

Hardened runtime controls such as immutable lock-down modes (-e 2) preventing attackers or administrators from altering rules or tampering with audit logs while the system is running.

Advanced event filtering, searching, and compliance reporting capabilities via companion utilities like ausearch and aureport using custom tracking identifiers (keys).

Auditd Examples

```
# Create a audit .rules files
sudo vim /etc/audit/rules.d/passwd.rules

# Check all implemented Audit Rules
sudo auditctl -l

# Reload auditd rules
sudo auditctl -R /etc/audit/rules.d/passwd.rules
sudo augenrules --load

# Add Rules to monitor changes in passwd and shadow file
-w /etc/passwd -p wa -k identity
-w /etc/shadow -p wa -k identity

# create rule for a specific binary or program
-a always,exit -F arch=b64 -S execve -F exe=/usr/bin/passwd -k passwd_execution

# search /var/log/audit/audit.log for identity logs
sudo ausearch -k identity
```

```
type=SYSCALL msg=audit(1791906066.038:2169): arch=c000003e syscall=82 success=yes exit=0 a0=7f0133e5e46a a1=7f0133e5e1ac a2=55afab22198d a3=55
aaf18dd1c0 items=5 ppid=13802 pid=13803 auid=1000 uid=0 gid=0 euid=0 suid=0 fsuid=0 egid=0 sgid=0 fsgid=0 tty=pts1 ses=3 comm="passwd" exe="/u
sr/bin/passwd" subj=unconfined_u:unconfined_r:passwd_t:s0-s0:c0.c1023 key="identity"
dev01@almaserver01:~$
```

Audit rules are defined using specific parameters to target system calls or monitor files. The filesystem watch rule structure uses -w to specify the target path or file, -p to define permissions to monitor such as write or attribute changes (wa), and -k to assign a custom search key for log filtering with ausearch. For low-level kernel tracking, system call rules use -a always,exit to trigger logs upon system call completion, -F arch=b64 to specify the system architecture, -S to target specific system calls like execve, and filters like -F exe= to restrict the rule to exact binaries, accompanied by a -k tracking key.

7. AIDE

Advanced Intrusion Detection Environment (AIDE) is a host-based file integrity checker designed to monitor and protect the baseline state of an operating system. Instead of tracking live actions like a logging daemon, AIDE acts as a digital fingerprinting utility that scans critical files and directories to build a cryptographic database snapshot. During subsequent system checks, it compares the current file attribute such as cryptographic hashes, file sizes, permissions, ownership, and SELinux context against this trusted baseline to instantly flag any unauthorized, silent, or malicious modifications.

Key capabilities of AIDE include:

Automated creation of a secure baseline database using strong hashing algorithms (such as SHA-256 or SHA-512) stored typically under /var/lib/aide/.

Comprehensive tracking of system file modifications, additions, or deletions across sensitive paths defined in /etc/aide.conf.

Deep attribute inspection verifying file permissions, modification times, inode changes, and SELinux security contexts.

Reliable detection of rootkits, trojans, and unauthorized binary replacements that standard log files might miss.Flexible database updates and checks via straightforward administrative commands (aide --init, aide --check, and aide –update).

```
# Install and verify installation of aide
sudo dnf install aide -y
aide -v

# Create a file in the root Directory
sudo touch /root/secret.txt
echo "important secret" | sudo tee /root/secret.txt

# Add a path to monitor
sudo vim /etc/aide.conf
/root/secret.txt R

# Create a initial Aide Database with file hashes etc. and rename it
sudo aide –init
sudo mv /var/lib/aide/aide.db.new.gz /var/lib/aide/aide.db.gz

# Change the file content and make a aide integrity check
aide –check
echo "added to file!" | sudo tee -a /root/secret.txt

# Update aide database after changes
sudo aide --update
```

```
# Ditto /var/log/sa reason...
!/var/log/httpd
# /boot/grub2/grubenv's timestamp is getting modified continuously due to "boot_success" implementation
!/boot/grub2/grubenv
# our lab test
/root/secret.txt FIPSR
-- INSERT --                                                                                                    359,23        Bot
```

```
dev01@alma10:~$ sudo mv /var/lib/aide/aide.db.new.gz /var/lib/aide/aide.db.gz
dev01@alma10:~$ sudo aide --check
Start timestamp: 2026-10-14 09:37:15 +0200 (AIDE 0.19.2)
AIDE found NO differences between database and filesystem. Looks okay!!

Number of entries:      135357
```

```
dev01@alma10:~$ echo "added to file!" | sudo tee -a /root/secret.txt
[sudo] password for dev01:
added to file!
dev01@alma10:~$ sudo aide --check
Start timestamp: 2026-10-14 09:45:19 +0200 (AIDE 0.19.2)
AIDE found differences between database and filesystem!!

Summary:
  Total number of entries:      135357
  Added entries:                0
  Removed entries:              0
  Changed entries:              1

---------------------------------------------------
Changed entries:
---------------------------------------------------

f > ... mc..H.... : /root/secret.txt
```

AIDE R Macro and Custom Rules Summary

What is the R Macro in AIDE?
In AIDE, "R" is a summary macro representing a group of file attributes. When you assign "R" to a path in the configuration file (`/etc/aide.conf`), such as `/root/secret.txt R`, it expands to a combination of multiple individual checks:
R = p + i + n + u + g + s + m + c + acl + selinux + xattrs + sha512
Note: Depending on the RHEL or AlmaLinux version, SHA-512 is typically used as the default cryptographic hash.

Specific File Properties Checked by R
When AIDE monitors a file using the R macro, it triggers an alert if any of the following properties change:

p (Permissions): File permissions were modified (e.g., via chmod).
i (Inode): The inode number changed, indicating the file was deleted and recreated.
n (Number of links): The count of hard links changed.
u (User): The file owner (UID) was changed (e.g., via chown).
g (Group): The group ownership (GID) was changed (e.g., via chgrp).
s (Size): The file size has changed.
m (Modification time): The time when the file content was last modified.
c (Change time): The time when the file metadata or attributes were modified.
acl / selinux / xattrs: Access Control Lists, SELinux security contexts, or extended attributes were altered.
sha512: The cryptographic content hash changed, ensuring that even if an attacker attempts to fake timestamps and file size, tampering is detected.

Comparison of Standard Macros

R (Read-only): Monitors everything (content, hashes, permissions, ownership, SELinux). Ideal for static system files like `/etc/passwd` or binaries in /usr/bin/.
L (Logfile): Monitors only permissions, ownership, and size while ignoring timestamps and hashes (useful for rapidly growing log files).
DATAONLY: Monitors content/hash and permissions while ignoring timestamps (mtime/ctime).
PERMS: Monitors only access rights (p), owners (u/g), SELinux, and ACLs, omitting file content checks.

Where to Define Custom Macros

Custom macros and rule definitions in AIDE are declared inside the main configuration file, typically located at /etc/aide.conf. They are usually placed near the top of the file before any selection lines are applied, or they can be defined within dedicated configuration snippets if the setup uses modular include paths (e.g., inside /etc/aide.d/).

AIDE Macro Syntax
CustomMacro = p + u + g + sha256
MyStrictRule = R + sha256

Ignoring Files and Attributes

Negative Selection Lines (Excluding Files/Directories)
To completely ignore a directory, file, or pattern so AIDE does not inspect it at all, prefix the path regular expression with an exclamation mark (!):

!/var/log/.*
!/tmp/.*
!/var/spool/mqueue

Selective Attribute Exclusion (Ignoring Specific Changes)

If you want to monitor a file or directory but ignore specific attributes (e.g., modification time or access time), you subtract those attributes using the minus sign (-) in a macro definition:

Ignore modification time (m) and change time (c) on a set of files
IGNORE_TIMES = R - m – c

Example: Ignore access time (atime / a) and check time
NO_ATIME = R – a

Restricting Matching with Equals Sign (=)

To monitor only a specific directory itself without recursively checking its contents or subdirectories, use the equals sign prefix (=):
=/etc/target_dir$ R

Common Ignore Syntax Examples in /etc/aide.conf

Include the /var directory with Logfile rules:
/var L

Exclude temporary and rapidly changing subdirectories within /var:

!/var/tmp/.*
!/var/cache/.*
!/var/log/app_debug\.log

8. Data, Storage, File Systems and Encryption

Storage

Modern solid-state storage, such as NVMe SSDs, relies on non-volatile NAND flash memory cells to store data electrically. Rather than dealing directly with these individual flash cells, the physical drive contains an integrated microcontroller known as the Flash Translation Layer (FTL). The kernel does not talk directly to every single raw memory cell or manage physical wear; instead, it communicates directly with this single storage microcontroller, which handles internal chip management, error correction, and wear leveling behind the scenes.

From the perspective of the operating system and kernel, the physical complexity is abstracted away, and the drive appears as a block device specifically, a linear, flat array of fixed-size storage blocks (typically 512 bytes or 4KB sectors). To handle input and output efficiently across modern multi-core processors, the kernel interacts with the drive using advanced queuing architectures, such as the high-performance submission and completion queues provided by NVMe, which allow parallel command processing without the bottlenecks of older storage protocols.

Partition tables

Partition tables like MBR and GPT are needed to divide a raw, continuous block device into logical sections called partitions. This division allows an operating system to organize data, isolate system files from user data, install multiple operating systems on a single drive, and provide the firmware with the exact starting and ending points required to boot the system.

MBR

The Master Boot Record is the legacy partitioning scheme introduced in the 1980s, which stores its partition information and bootstrap code in the very first sector of the disk. MBR is severely constrained by modern standards: it relies on 32-bit sector addressing, limiting support to a maximum disk size of 2 terabytes, and restricts a drive to only 4 primary partitions unless complex extended and logical partition workarounds are used. Furthermore, because MBR keeps its single partition table in just one location at the beginning of the drive, any physical damage or corruption to that sector can render the entire storage medium inaccessible.

GPT

The GUID Partition Table is the modern standard designed to replace MBR, operating alongside UEFI firmware to eliminate its predecessors restrictions. GPT uses 64-bit logical block addressing, supporting massive drives up to several zettabytes and allowing a practically limitless number of partitions, typically capped at 128 in operating systems like Windows. For robust reliability, GPT stores a primary partition table at the beginning of the disk and a crucial backup copy at the very end of the disk, utilizing cyclic redundancy check checksums to actively detect corruption and automatically restore damaged metadata if necessary.

GPT Structure

The structure of a GUID Partition Table is organized across specific logical block addresses to ensure functionality and redundancy. Sector 0 contains a protective Master Boot Record for legacy compatibility to prevent old disk utilities from overwriting the drive. Sector 1 holds the primary GPT header, storing critical metadata such as the disk GUID, header locations, usable data sectors, and cyclic redundancy check checksums. Sectors 2 through 33 contain the partition entry array, holding up to 128 partition entries where each entry takes 128 bytes. Each partition entry stores metadata including the partition type GUID, unique partition GUID, starting and ending logical block addresses, attribute flags, and a Unicode partition name. A complete duplicate of this metadata structure is mirrored at the very end of the disk for fault tolerance. When partitions are modified, partitioning tools update the metadata, recalculate the checksums, and write the updated structures to both the primary and backup locations to keep both copies synchronized.

Filesystem

A filesystem is a digital filing system that controls how an operating system names, stores, and retrieves data on a drive. Instead of viewing raw, unorganized storage blocks, the operating system uses the filesystem to organize data into clean folders and files.

The fourth extended file system (ext4) is the default filesystem for many Linux distributions. To keep things organized and performant, ext4 splits a partition into smaller units called block groups. Each block group contains a superblock copy (main settings), bitmaps to track which blocks and inodes are free, an inode table holding file details, and data blocks where file contents are stored. A filesystem works directly with a partition table like MBR or GPT, as it is formatted inside a partition boundary. The partition table tells the kernel where the ext4 filesystem starts and ends on the drive.

Every file or folder in ext4 is managed by an inode (index node). An inode is a small metadata record that stores file permissions, file size, owner User ID, Group ID, timestamps, and pointers to the data blocks holding the actual contents. Importantly, the inode does not store the filename itself. Filenames are kept separately in directory blocks as simple lists that link names to their matching inode numbers.

For security, standard Linux permissions (read, write, execute) and owner IDs are saved inside the inode. Advanced security frameworks like SELinux store security labels (user:role:type:level) in extended attributes. In ext4, these SELinux labels are stored directly inside the inode or in linked extended attribute blocks, allowing the kernel to check access rights instantly whenever a file is opened.

Hard and Soft Links

A hard link is a direct reference to the same inode and data blocks as the original file. Because a hard link shares the exact same inode number, modifying the content through one link updates it for all of them. The file data is only permanently deleted when all hard links and the original file are removed (meaning the link count drops to zero). Hard links cannot cross different filesystems or partitions because inode numbers are only unique within a single filesystem.

A soft link, also known as a symbolic link or symlink, is a special file that acts as a shortcut pointing to the path of the target file rather than its inode. A soft link has its own unique inode and its own data block, which simply stores the text string of the target file's path. If the original file is deleted or moved, the soft link breaks and becomes a dangling link because the target path no longer exists. Unlike hard links, soft links can easily span across different filesystems and directories.

```
# Show inode number
ls -i filename

# Create a Hardlink
ln original_file.txt hard_link.txt

# Create a Softlink
ln -s original_file.txt soft_link.txt

# Find all files associated with a specific inode number
find /path/to/search -inum 12345678

# More informations about disk, partitions and filesystems
sudo blkid
lsblk
sudo fdisk -l
sudo dumpe2fs
df -h
```

Data encryption

Lab Scenario: LUKS and Network-Based Device Encryption (NBDE)

In this lab, we are setting up a secure storage environment using LUKS for local disk encryption combined with NBDE (Network-Based Device Encryption) for automated, secure unlocking during system boot.

alma10: The client machine where we configure a dedicated 10GB virtual disk with LUKS local encryption. We will use Clevis on this client to automatically unlock the encrypted volume over the network.

almaserver01: The primary Tang server. It provides network-based binding services so the client can retrieve its decryption keys securely over the network without manual password entry.

almaserver02: The secondary Tang server. This server acts as a redundancy node to implement high availability, fault tolerance, and threshold-based encryption (Shamir's Secret Sharing) for the client's encrypted storage.

LUKS

Linux Unified Key Setup is the standard disk encryption specification for Linux systems. It provides a secure, transparent way to encrypt block devices, such as hard drives, partitions, or logical volumes.

When you format a device with LUKS, all data written to it is automatically encrypted on the fly, and all data read from it is transparently decrypted in real-time by the Linux kernel, provided you have the correct credentials. LUKS stores all necessary setup metadata (such as encryption algorithms, key slots, and salts) directly in a header at the beginning of the storage device. This allows you to manage multiple passphrases or key files for the same encrypted container securely and flexibly.

Tang

Tang is a lightweight, stateless network binding server designed specifically to anchor cryptographic keys to a network presence. Operating as a core component of a Network-Based Device Encryption (NBDE) framework, Tang provides a secure mechanism for client systems to retrieve decryption keys or perform cryptographic key exchanges automatically during the system boot sequence, removing the requirement for manual passphrase entry.

Clevis

Clevis is a client-side, pluggable automated decryption framework designed to work seamlessly with LUKS-encrypted block devices and various trust anchors, such as network-based Tang servers or hardware-based TPM2 chips. It enables secure, unattended unlocking of encrypted storage during the early system boot process without requiring manual user intervention.

Clevis utilizes modular components known as "pins" to define the mechanism used to encrypt and decrypt cryptographic secrets. Common pins include tang (for network-bound binding), tpm2 (for hardware-based binding using a Trusted Platform Module), and sss (Shamir's Secret Sharing for high-availability multi-server setups).

Clevis includes hooks for **dracut (clevis-dracut)** to ensure compatibility with the initial RAM disk. During the system startup sequence, the initramfs environment executes the Clevis client routines to reach out over the network (or query hardware), reconstruct the decryption secret, unlock the encrypted storage device, and allow the boot process to proceed normally.

```
# Initialize Luks Container
sudo cryptsetup luksFormat /dev/nvme0n2

# Decrypt Luks Container
sudo cryptsetup luksOpen /dev/nvme0n2 secure_vol
```

sudo cryptsetup luksFormat /dev/nvme0n2

What it does:

It initializes a new LUKS container on the disk /dev/nvme0n2.
It overwrites the first sectors of the disk with a LUKS header.
It generates a random master key (used to encrypt the actual data).
It prompts you to set a passphrase. This passphrase encrypts the master key so it can be securely stored in the header.

sudo cryptsetup luksOpen /dev/nvme0n2 secure_vol

What it does:

It opens (unlocks) the encrypted container and makes it usable.
You enter your passphrase, which decrypts the master key and loads it into the kernel memory. Linux then creates a virtual, decrypted block device under the name secure_vol you chose, located at /dev/mapper/secure_vol. You can subsequently create a file system (such as XFS) on this virtual device and mount it.

```
# Install tang server
sudo dnf install tang -y

# Enable and start and verify the tang deamon
sudo systemctl enable --now tangd.socket
sudo systemctl status tangd.socket

# Open Firewall Port
sudo firewall-cmd --add-service=http --permanent
sudo firewall-cmd --reload

# Install clevis packets
sudo dnf install -y clevis clevis-luks clevis-dracut

# Bind Container with the tang server
sudo clevis luks bind -d /dev/nvme0n2 tang '{"url":"http://almaserver01"}'

# update initramfs
sudo dracut -f


# update initramfs
sudo clevis luks bind -d /dev/nvme0n2 tang
'{"t":1,"pins":{"tang":[{"url":"http://almaserver01"},{"url":"http://almaserver02"}]}}'
```

```
dev01@alma10:~$ sudo clevis luks bind -d /dev/nvme0n2 tang '{"url":"http://almaserver01"}'
[sudo] password for dev01:
Enter existing LUKS password:
The advertisement contains the following signing keys:

U-4K31Y6AQ5DjBkaGtAfqwNfwSgh2e8zsIGiu20REzA

Do you wish to trust these keys? [ynYN] y
dev01@alma10:~$
```

Installing Clevis Packages

On the client machine, we install the required tools for Clevis and its integration into the early boot process.

Tang Setup

Binding the LUKS Container to the Tang Server (clevis luks bind) We link our encrypted container (/dev/nvme0n2) with our Tang server (almaserver01). During this command, you are prompted to enter your current LUKS passphrase. Clevis then generates a new cryptographic key, coordinates with the Tang server to secure it, and adds it as a new key slot in your LUKS container, establishing the network-based unlock mechanism.

Updating the Initramfs (dracut)

Since the client needs to unlock the drive very early during system startup (over the network before the main operating system and standard network services start), these Clevis components must be embedded into the initial RAM disk.

This ensures that the initramfs environment contains all necessary network drivers and Clevis hooks to perform the automated decryption successfully during the boot sequence.

Configuring SSS (Shamir's Secret Sharing) Policy on the Client (client1)

To achieve high availability and prevent a single point of failure, we bind our LUKS container to multiple Tang servers using Clevis's SSS pin. We define a threshold (e.g., "t:1"), meaning that at least 1 out of the 2 specified Tang servers must be reachable to unlock the device.
This provides fault tolerance. If one Tang server fails or goes offline for maintenance, the client can still successfully retrieve its keys and boot automatically via the remaining operational server.

Note: While we use unencrypted HTTP for simplicity in this lab environment, production environments must secure traffic using HTTPS with TLS certificates to prevent eavesdropping, man-in-the-middle attacks, or key interception over the network.

9. OpenScap

OpenSCAP is a powerful, modular command-line framework and standard toolset used across enterprise Linux environments to assess, measure, and enforce system security and compliance. It enables administrators and security auditors to evaluate operating systems against industry-recognized security benchmarks and compliance standards, such as CIS benchmarks, DISA STIGs, HIPAA, and BSI guidelines.

At its core, OpenSCAP operates using SCAP (Security Content Automation Protocol) data streams comprehensive XML databases provided by projects like ComplianceAsCode (e.g., ssg-almalinux10-ds.xml). These data streams bundle official security rules, checklists, and OVAL definitions tailored specifically for the target operating system.

The OpenSCAP ecosystem provides a robust set of capabilities:

Compliance Evaluation and Scanning: Using the oscap xccdf eval command, the tool scans running systems against defined security profiles to identify configuration drift, security gaps, and non-compliant settings. It handles everything from file permissions and SSH configurations to kernel parameters.

Reporting and Export Formats: OpenSCAP converts raw technical data into human-readable, interactive HTML reports containing detailed rule statuses (pass, fail, error), rule descriptions, and manual remediation steps. For enterprise integrations and central management dashboards, it supports machine-readable XML results as well as standardized ARF (Asset Reporting Format) files.

Vulnerability and CVE Scanning: Beyond static system configurations, OpenSCAP can evaluate systems against OVAL-based vulnerability data streams to detect unpatched software flaws and known CVEs.

Profile Tailoring: The framework allows administrators to customize existing security baselines or exclude specific rules to match internal corporate compliance policies without altering the underlying master benchmark files.

Remediation Modes and Fix Generation: OpenSCAP provides versatile approaches to resolving identified security failures:

Online Remediation: Applies security fixes directly to the system during the live compliance scan.

Offline Remediation: Processes existing result files post-scan to correct non-compliant settings.

Fix Generation: Extracts remediation logic from scan results and translates failing rules into automated fix mechanisms. Through the fix generation feature, OpenSCAP can output executable Bash scripts, Puppet manifests, or Ansible playbooks.

```
# Install the scanning engine and the rule book database
sudo dnf install -y openscap-scanner scap-security-guide

# Check for the rules xml database get informations about scanning profiles and rules
ls -l /usr/share/xml/scap/ssg/content/
oscap info /usr/share/xml/scap/ssg/content/ssg-almalinux10-ds.xml

# Scan with the cis ruleset and create reports
sudo oscap xccdf eval \

  --profile xccdf_org.ssgproject.content_profile_cis_server_l1 \

  --results results.xml \

  --report report.html \

  /usr/share/xml/scap/ssg/content/ssg-almalinux10-ds.xml

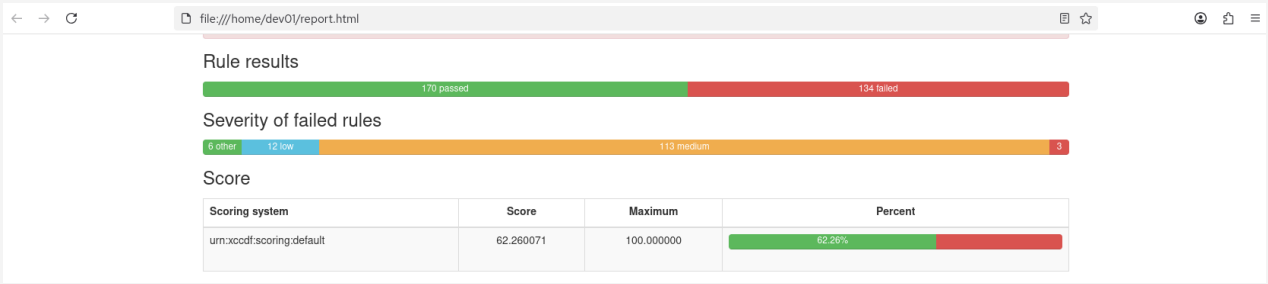
# Subsequent report generation
oscap xccdf generate report results.xml > report-new.html

# Show results in the terminal
oscap xccdf info results.xml

# Subsequent remediation
oscap xccdf remediate results.xml

# Perform a CVE scan
sudo oscap oval eval \
 --results cve-results.xml \
 --report cve-report.html \
 /path/to/oval-definition-file.xml

# Create automaticly ansible remediation playbook
oscap xccdf generate fix \
  --fix-type ansible \
  --profile xccdf_org.ssgproject.content_profile_cis_server_l1 \
  --output remediation-playbook.yml \
  results.xml
```



To inspect available security profiles and benchmarks within the data stream, run oscap info /usr/share/xml/scap/ssg/content/ssg-almalinux10-ds.xml. This lists all supported standards such as CIS and DISA STIG alongside their exact profile IDs.

To perform a live compliance scan and generate both machine-readable results and a human-readable web report, use sudo oscap xccdf eval --profile xccdf_org.ssgproject.content_profile_cis_server_l1 --results results.xml --report report.html /usr/share/xml/scap/ssg/content/ssg-almalinux10-ds.xml.

To check the metadata of an existing result file without opening the full report, run oscap xccdf info results.xml. If you need to generate a standalone HTML report later from existing XML data, use oscap xccdf generate report results.xml > report-neu.html.

To check a system for unpatched software vulnerabilities and known CVEs using OVAL definitions, run sudo oscap oval eval --results cve-results.xml --report cve-report.html /path/to/oval-definition-file.xml. Note that OpenSCAP identifies vulnerabilities through OVAL but does not perform direct package patching; updates must still be handled via package managers like dnf or configuration tools.

Regarding remediation and automated hardening, OpenSCAP offers multiple options. You can perform offline remediation based on scan results using sudo oscap xccdf remediate results.xml, or you can generate automation assets using oscap xccdf generate fix --fix-type ansible --profile xccdf_org.ssgproject.content_profile_cis_server_l1 --output remediation-playbook.yml results.xml to translate failing rules into a reusable Ansible playbook.

Crucial Pitfall Warning for Production Environments: Never blindly execute automated remediation or online hardening scripts directly on production systems without prior testing in a staging environment. Aggressive security baselines like CIS or STIG modify core system settings, strict file permissions, firewall rules, and kernel parameters, which can inadvertently break running enterprise applications, disrupt network services, or lock out administrators. Always evaluate scan reports first, test generated Ansible playbooks in non-production environments, and carefully review changes before enforcing them across production infrastructure.